

This assignment has in total 100 base points and 15 extra points, and the cap is 100. Bonus questions are indicated using the ★ mark.

Submission Instructions: Please submit to Gradescope. During submission, you need to **mark/map the solution to each question**; otherwise, we may apply a penalty.

Another notice: you only get full credits for the algorithm design questions if your algorithm matches the desired complexity. If the desirable complexity is not stated, you need to design an algorithm to be as fast as possible.

Please specify the following information before submission:

- Your Name: Yixia Yu
- Your NetID: yy5091

Problem 1: Bottleneck shortest paths [8 + 12 pts]

Let $G = (V, E)$ be a connected undirected graph and $w : E \rightarrow \mathbb{R}$ be a weight function. Write $n = |V|$ and $m = |E|$. Consider a path $\pi = (v_0, v_1, \dots, v_r)$ in G . We define the *bottleneck length* of π as $\max_{i=1}^r w(v_{i-1}, v_i)$, i.e., the maximum weight of an edge on π . A *bottleneck shortest path* from s to t refers to a path with the smallest bottleneck length. The *bottleneck distance* from s to t , denoted by $b(s, t)$, is the bottleneck length of a bottleneck shortest path from s to t . Given a source vertex $s \in V$ and a target vertex $t \in V$, we want to compute $b(s, t)$. We solve this problem via the following two steps.

- (1) Suppose $E = \{e_1, \dots, e_m\}$ where $w(e_1) \leq \dots \leq w(e_m)$, and define $G_i = (V, E_i)$ for $E_i = \{e_1, \dots, e_i\}$. Show that for any $v \in V \setminus \{s\}$, $b(s, v)$ is equal to $w(e_i)$ where $i \in \{1, \dots, m\}$ is the smallest index such that v and s lie in the same connected component of G_i .
- (2) Based on the result of (a), design an $O((n+m) \log n)$ -time algorithm $\text{BOTTLENECK}(G, w, s, t)$ that returns the bottleneck distance $b(s, t)$ in G (with respect to the weight function w). Describe the basic idea of your algorithm and give the pseudocode. Briefly justify its correctness.

Solution. Please write down your solution to Problem 1 here.

Problem 2: Approximate shortest paths [8 + 12 pts]

The purpose of this problem is to design (efficient) approximation algorithms for the single-source shortest-path (SSSP) problem in a graph whose edge weights are in a fixed range. For convenience, we only consider connected undirected graphs. We divide the problem into the following two steps.

- (1) Let $G = (V, E)$ be a connected undirected graph with a weight function $w : E \rightarrow \{1, \dots, 10000\}$ (i.e., the weight of each edge of G is an integer between 1 and 10000). Write $n = |V|$ and $m = |E|$. Design an *exact* algorithm $\text{SSSP}(G, w, s)$ with time complexity $O(n + m)$ to solve the SSSP problem on G with weight function w and source vertex $s \in V$. Your algorithm should return an array D such that $D[t]$ is the shortest-path distance from s to t in G . Describe the basic idea of your algorithm and give the pseudo-code. Briefly justify its correctness.

(**Hint.** Note that the known algorithms for general SSSP cannot solve the problem in $O(n + m)$ time. So here you have to exploit the property that the weights are small integers. Try to first transform the graph G into another graph in which all edges have weight 1, and solve the SSSP problem in the new graph.)

- (2) Based on the result of (a), we now solve SSSP on graphs with *real* weights (in a fixed range). Let $G = (V, E)$ be a connected undirected graph with a weight function $w : E \rightarrow [1, 2]$ (i.e., the weight of each edge of G is a real number between 1 and 2). Write $n = |V|$ and $m = |E|$. Design a 1.01-approximation algorithm $\text{APPRXSSSP}(G, w, s)$ with time complexity $O(n + m)$ for SSSP on G with weight function w and source vertex $s \in V$. That is, your algorithm should return an array D such that $d(s, t) \leq D[t] \leq 1.01 \cdot d(s, t)$, where $d(s, t)$ is the shortest-path distance from s to t in G . Describe the basic idea of your algorithm and give the pseudo-code. Prove that it is truly a 1.01-approximation algorithm.

(**Hint.** Try to modify the weights of the edges a little bit and properly apply the result of (a) to solve the problem on the modified graph.)

Solution. Please write down your solution to Problem 2 here.

You can continue your solution here.

Problem 3: Approximation for knapsack [15 pts]

Consider an easy variant of the 0-1 knapsack problem in which we want to include items in a knapsack to make the knapsack *as full as possible*. Formally, given a number $W \in \mathbb{R}_{\geq 0}$ that represents the capacity of the knapsack and n numbers $w_1, \dots, w_n \in \mathbb{R}_{\geq 0}$ that represent the weights of n items, our goal is to compute a subset $P \subseteq \{1, \dots, n\}$ satisfying $\sum_{i \in P} w_i \leq W$ such that $\sum_{i \in P} w_i$ is maximized (here the set P indicates the items you want to include in the knapsack). Note that W and $w_1, \dots, w_n$ are all (non-negative) real numbers, and thus the dynamic-programming algorithm for 0-1 knapsack does not work. Instead, we are interested in approximation algorithms for the problem. Design a $\frac{1}{2}$ -approximation algorithm $\text{APPRXKNAPSACK}(W, w_1, \dots, w_n)$ for the problem with time complexity $O(n \log n)$. That is, the algorithm should return a feasible solution $P \subseteq \{1, \dots, n\}$ such that $\sum_{i \in P} w_i \geq \frac{1}{2} \cdot \text{opt}$, where $\text{opt} = \sum_{i \in P^*} w_i$ for an optimal solution $P^* \subseteq \{1, \dots, n\}$. Describe the basic idea of your algorithm and give the pseudo-code. Prove that your algorithm is truly a $\frac{1}{2}$ -approximation algorithm.

(**Hint.** Try to use some greedy strategy to select the items.)

Solution. **Please write down your solution to Problem 3 here.**

Problem 4*: Another algorithm for farthest pair [15* pts]

Recall that in the farthest-pair problem, we are given a set $S = \{a_1, \dots, a_n\}$ of points in $\mathbb{R}^d$ and the goal is to find two points in S that are farthest to each other. Consider the following algorithm, which is a variant of the $\frac{1}{2}$ -approximation algorithm you have seen in the lecture.

Algorithm 1 FARTHESTPAIR($S = \{a_1, \dots, a_n\}$)

```

 $A[1] \leftarrow a_1$ 
 $i \leftarrow 1$ 
while true do
     $p \leftarrow$  the point in  $S$  farthest to  $A[i]$ 
     $i \leftarrow i + 1$ 
     $A[i] \leftarrow p$ 
    if  $A[j] = p$  for some  $j \in \{1, \dots, i - 1\}$  then break
    end if
end while
return ( $A[i - 1], A[i]$ )

```

Roughly speaking, the above algorithm starts from the point a_1 , and in each step moves to the point in S farthest to the current point. The procedure terminates when it moves to a point that has been visited previously. When it terminates, the algorithm simply returns the pair of points visited in the last two steps as its output.

Find the largest number $\rho < 1$ such that this algorithm is a ρ -approximation algorithm for the farthest-pair problem. Prove the correctness of your answer. That is, for the number ρ you find, show that (i) the algorithm is truly a ρ -approximation algorithm and (ii) for any $\rho' > \rho$, the algorithm is *not* a ρ' -approximation algorithm. Note that the algorithm works for the problem in $\mathbb{R}^d$ for a general $d \geq 1$, not just in the plane.

Solution. **Please write down your solution to Problem 4 here.**

Problem 5: Reductions [5 + 10 pts]

- (1) Show that the weighted maximum-cut problem (partition the vertices of a graph into two subsets, and maximize the sum of the weights of the crossing edges) can be reduced to the partition problem (partition a set of positive integers into two subsets such that the sums of the two are equal.)
- (2) Prove that Pos-or-Neg 3-SAT is NP-hard using reduction. Pos-or-Neg 3-SAT refers to a special case of 3-SAT where every clause either consists of all positive variables or all negations of variables.

Solution. **Please write down your solution to Problem 5 here.**

Problem 6: Vertex Weighted SSSP [10 pts]

Let $G = (V, E)$ be a simple undirected graph with a weight function $w : V \rightarrow \mathbb{R}_{>0}$ on its vertices. The *length* of a path $(v_0, v_1, \dots, v_r)$ in G is defined as $\sum_{i=0}^r w(v_i)$. Design an $O(n^2)$ -time algorithm to solve the SSSP problem in this setting.

Solution. **Please write down your solution to Problem 6 here.**

Problem 7: Facility Location Function [10 pts]

Recall that the facility location function is defined as

$$f(A) = \sum_{v \in V} \max_{a \in A} \text{sim}(v, a)$$

A feature-based submodular function can be of the form

$$f(A) = \sum_j w_j \phi_j \left(\sum_{v \in V} m_j(v) \right)$$

Here ϕ_j is any concave function, and w_j is a scalar. Show that the facility location function can be written as a feature-based submodular function. The outer sum should be over a number that is polynomial in n , i.e., the choices of j should be a polynomial of n .

Solution. **Please write down your solution to Problem 6 here.**

Problem 8: Matroid [10 pts]

Given a tree $T = (V, E)$ and a subset of vertices A . Let $h(E')$, $E' \subseteq E$, be the number of connected components in the forest $T \setminus E'$. Define a set of subset of edges as

$$\mathcal{I} = \{E' \subseteq E \mid h(E') = |E'| + 1\}$$

Prove that $\mathcal{M} = (E, \mathcal{I})$ is a matroid.